

Document Extraction Service with a Fine-Tuned Small VLM

Role: AI Engineer (3+ years)

Time allowed	7 days from receipt
Hardware	Free Colab / Kaggle GPU (T4, 16 GB) is sufficient
Submission	GitHub repository link + Hugging Face adapter link
Questions	Reply to the email this was sent with

01 The Goal

Build a small service that accepts a **PDF upload** and returns **structured, validated JSON** in the same HTTP response. Extraction must be done by a **small Vision Language Model (VLM)** that you fine-tune yourself, reading page images directly.

What matters most to us: the service must never return a value that is not actually in the document. When it is unsure, it should return `null` or flag the field, not guess.

Flow at a glance

PDF upload → **render pages to images** → **fine-tuned VLM** → **JSON validation + grounding checks** → **response**

02 Dataset and Output Schema

Use **CORD v2** (receipts with labelled fields), available on Hugging Face. You may use SROIE or DocILE instead if you explain why. Keep a held-out test set that is never used for training.

Your output must follow this schema:

```
{
  "store_name": string | null,
  "date": string | null,           // ISO format: YYYY-MM-DD
  "line_items": [
    { "name": string, "qty": number | null,
      "unit_price": number | null, "amount": number | null }
  ],
  "subtotal": number | null,
  "tax": number | null,
  "total": number | null,
  "_verification": { "<field>": "verified" | "unverified" | "not_found" }
}
```

03 Tasks

Task A · Fine-tune a small VLM

- Pick a model with **4B parameters or fewer** (e.g. Qwen2.5-VL-3B, SmolVLM, PaliGemma 2, Florence-2) and justify the choice.
- Fine-tune with LoRA or QLoRA to produce the JSON schema from page images.
- Choose an image resolution and explain the accuracy vs. memory trade-off.
- Report **zero-shot vs. fine-tuned** results on the same test set.

Task B · Structured output and hallucination control

- Every response must be valid JSON that passes schema validation (e.g. Pydantic).
- Add a **grounding step** that verifies each value against the document, for example bounding-box crops checked with the PDF text layer or a lightweight OCR.
- Unverified values become null or are marked unverified.
- Add sanity checks such as *line items sum to subtotal* and *subtotal + tax = total*.

Task C · Synchronous API that runs one job at a time

- POST /extract accepts a PDF and returns the JSON in the same response. Multi-page PDFs are merged into one result.
- Load the model **once** at startup.
- Only **one inference job** may run on the GPU at a time. Decide what happens to other requests (queue, timeout, or 429/503) and explain why.
- Include a script that sends **5 parallel requests** and shows the behaviour.
- Handle bad input: file too large, too many pages, corrupt or password-protected PDFs, and timeouts.
- Provide a Dockerfile.

Task D · Feasibility report (1–2 pages)

- Is a small VLM a good fit for production extraction? Answer with numbers: accuracy, latency per page, GPU memory, and estimated cost per 1,000 pages.
- Describe where it fails: dense tables, poor scans, rotation, small fonts, multi-page documents.
- *Optional*: compare against an OCR + text-only model pipeline.

04 Metrics to Report

Metric	What it tells us
Field-level precision, recall, F1	Overall extraction quality, per field
Exact match rate	How often a full field is exactly right
JSON validity rate	Should be 100%
Hallucination rate	Share of non-null values not found in the document, before and after grounding
Latency p50 / p95 per page	Speed of the service
Peak GPU memory	Deployment cost and feasibility

05 What to Submit

Deliverable	Details
GitHub repository	Clean structure, with a README covering setup, design decisions and trade-offs
Model adapter	LoRA weights on Hugging Face, plus the training notebook or script
Evaluation report	Metric tables for zero-shot vs. fine-tuned
Feasibility report	Task D, 1–2 pages
Concurrency test	Script and its captured output
Demo video (optional)	5 minutes or less

06 How We Will Evaluate

Area	Weight
VLM fine-tuning quality and baseline comparison	25%
Grounding and hallucination handling	25%
API design and one-job-at-a-time handling	20%
Feasibility report backed by real numbers	15%
Evaluation rigour and code quality	15%

07 Rules

- Paid LLM APIs (GPT-4o, Claude, Gemini, etc.) must **not** be used as the extraction model. If you use any for data preparation, say so in the README.
- Open-source libraries and AI coding assistants are allowed. You must be able to explain every part of your code in the next round.
- Be honest about limitations. A smaller, working and well-measured solution scores higher than an ambitious one that does not run.

Next step: shortlisted candidates will walk us through their solution in a technical discussion. Good luck, and we look forward to seeing your work.